

CMPE 275 – Mini Project 1

Memory Layout and Parallel Query Optimization for TLC Trip Data

Alekya Gudise
Michael Kennedy

1. Introduction

Modern data processing systems must efficiently analyze large datasets containing millions or billions of records. Achieving high performance requires careful consideration of both data layout in memory and parallel execution strategies.

Traditional object-oriented designs frequently employ an Array of Structures (AoS) layout, where each record contains all attributes of an entity. While intuitive, this layout often performs poorly for analytical workloads where queries operate on only a subset of attributes. In contrast, a Structure of Arrays (SoA) layout stores each attribute in a separate contiguous array, improving memory locality and enabling better cache utilization.

This project investigates the performance implications of data layout and parallelism when executing analytical queries on the New York City TLC High Volume For-Hire Vehicle (HVFHV) dataset. The dataset contains over 94 million trip records for the year 2023 using months 1-5.

Three implementations were developed and evaluated:

1. Phase 1: Serial query execution using AoS
2. Phase 2: OpenMP-parallel queries using AoS
3. Phase 3: OpenMP-parallel queries using SoA with query-driven data loading

The objective of this study is to evaluate how memory layout and parallelism affect query performance and scalability.

2. Dataset

The experiments use the NYC TLC High Volume For-Hire Vehicle Trip Data for 2023.

The dataset includes fields such as:

- pickup location
- dropoff location
- trip distance
- trip time
- base passenger fare

- driver pay
- tips
- flags for ride sharing and accessibility

Each record represents a single trip.

For this experiment, the dataset used was a CSV in order to enable simple efficient parsing without using libraries.

Only the first five months of 2023 were used in order to remain within the project's dataset size constraints.

Key dataset properties:

Property	Value
Dataset	NYC TLC HVFHV
Year	2023 (1-5)
Records	~94 million
File Size	~14GB CSV
Attributes per record	24

Source:

New York City Taxi & Limousine Commission. (2023). High Volume For-Hire Vehicle Trip Records.

<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

3. Query Workload

Three analytical queries were evaluated:

1. Tips Query

```
count trips where tips > 5
```

2. Fare Query

```
count trips where base_passenger_fare > 40
```

3. Distance Query

```
count trips where trip_miles > 10
```

These queries represent typical analytical operations where only one column is needed for filtering.

4. System Design

4.1 Phase 1 – Array of Structures (Serial)

Phase 1 stores each record using a structure representing a trip:

```
struct TripRecord {  
    double trip_miles;  
    double base_passenger_fare;  
    double tips;  
    ...  
};
```

All records are stored in a vector:

```
std::vector<TripRecord>
```

Queries iterate over the vector and inspect relevant fields.

However, because each structure contains many attributes, memory accesses bring unnecessary data into cache.

4.2 Phase 2 – AoS with OpenMP Parallelization

Phase 2 parallelizes the queries using OpenMP:

```
#pragma omp parallel for reduction(+:count)
```

Multiple threads divide the dataset and process records concurrently.

However, because the AoS layout remains unchanged, each thread still loads full structures containing unused fields.

4.3 Phase 3 – Structure of Arrays (SoA)

Phase 3 reorganizes the dataset into a columnar format:

```
struct TripTable {  
    std::vector<double> tips;
```

```

std::vector<double> fare;
std::vector<double> miles;
};

```

Only columns required by the queries are loaded into memory.

Benefits:

- improved cache locality
- contiguous memory access
- reduced memory bandwidth usage
- improved vectorization

Parallel queries are then executed with OpenMP.

5. Experimental Methodology

All experiments were conducted using a dedicated system to ensure consistency.

Component	Specification
CPU	Apple Silicon
Cores	12 (8 performance + 4 efficiency)
RAM	32GB
OS	macOS
Threads tested	12
Dataset	2023 HVFHV (1-5)
Runs per experiment	10
Measurement tool	/usr/bin/time

Each experiment measured:

- data load time
- query execution time
- memory usage
- CPU time

Warmup runs were performed to reduce the effects of disk caching and initialization overhead.

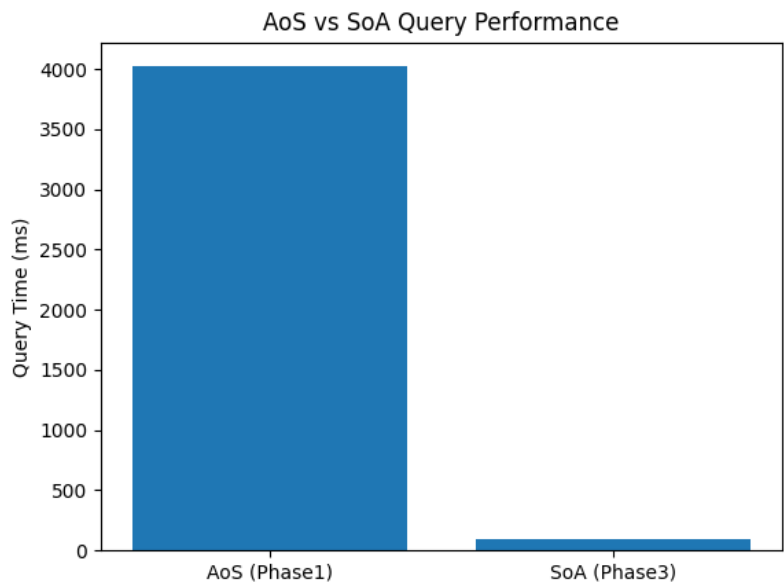
6. Experimental Results

6.1 AoS vs SoA Query Performance

The SoA implementation achieved a dramatic improvement in query performance.

This represents approximately a 45× performance improvement.

The improvement arises because SoA stores queried columns contiguously, enabling efficient cache utilization and minimizing unnecessary memory accesses.



As shown in the figure, reorganizing the data from an Array of Structures to a Structure of Arrays dramatically improved query performance. The AoS implementation required approximately 4000 ms to execute the tips query, whereas the SoA implementation completed the same query in approximately 89 ms, representing roughly a 45× performance improvement.

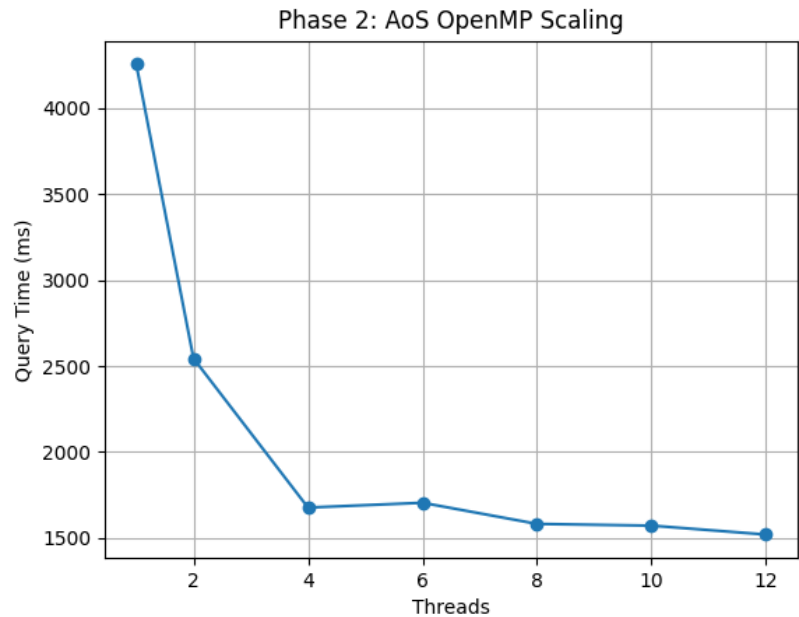
Implementation	Query Time
AoS	~4000ms
SoA	~89ms

6.2 Phase 2 OpenMP Scaling

The AoS implementation shows limited scalability.

The figure illustrates how query performance changes as the number of OpenMP threads increases when using the AoS data layout. Speedup plateaued after approximately four threads. This plateau indicates that the workload becomes memory bandwidth bound, meaning additional threads compete for access to main memory rather than improving compute throughput.

This behavior suggests that the system became memory bandwidth limited, as each thread loads large structures containing unused fields.



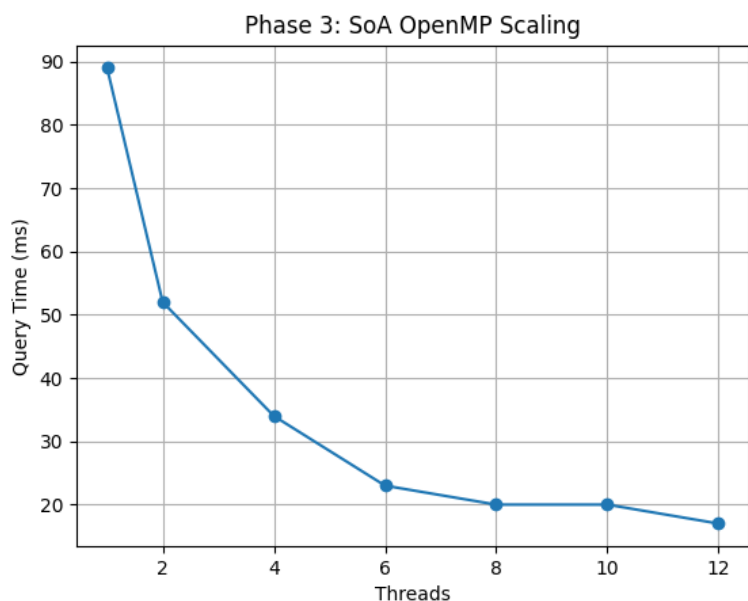
6.3 Phase 3 OpenMP Scaling

The SoA implementation demonstrated significantly better scaling.

Performance continued improving up to 12 threads, achieving over 5× speedup.

The improvement occurs because each thread processes contiguous arrays with minimal cache overhead.

As shown in the figure, the SoA layout scales significantly better than the AoS implementation. Query execution time decreases steadily as additional threads are used.



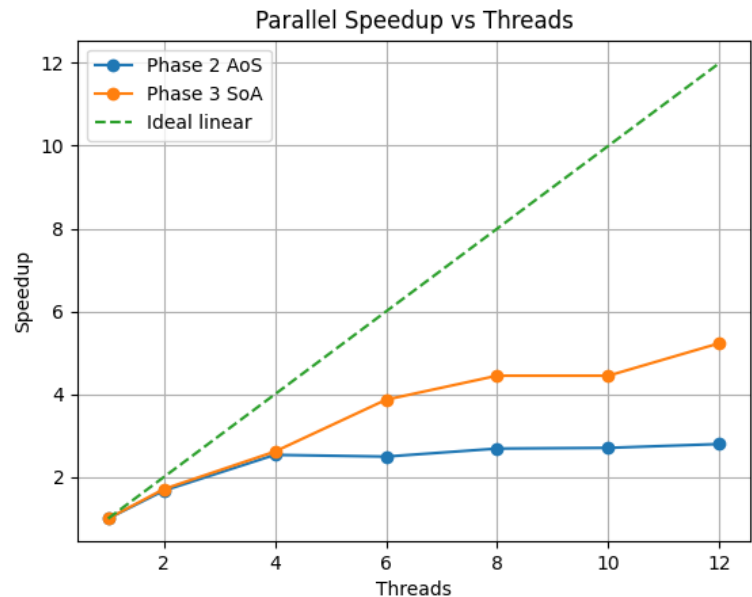
6.4 Parallel Speedup

The speedup graph compares the measured performance to ideal linear scaling.

Phase 2 achieves approximately: $2.8\times$ speedup with 12 threads

Phase 3 achieves approximately: $5.2\times$ speedup with 12 threads

Although neither implementation reaches ideal linear scaling, the SoA layout substantially improves parallel efficiency.



6.5 Single Thread Performance

The most significant result of the experiment is that memory layout optimization alone dramatically improves performance, even without parallelism.

Phase 3 reduced the single-thread query time from approximately 4 seconds to under 100 milliseconds.

7. Discussion

The results demonstrate several important principles of high-performance data processing.

First, data layout has a major impact on performance. The AoS representation forces the processor to load entire records even when only one field is required. This increases memory traffic and reduces cache efficiency.

Second, parallelism alone cannot compensate for poor memory layout. Although OpenMP improved performance in Phase 2, scalability quickly plateaued because threads compete for memory bandwidth.

Third, columnar data layouts enable both cache efficiency and improved parallel scaling. By storing attributes in contiguous arrays, the SoA implementation significantly reduced memory access overhead and allowed threads to process data more efficiently.

These findings align with the design principles used in modern analytical databases and columnar storage engines. Column-oriented storage has become the foundation of many modern analytical systems such as Apache Parquet and Apache Arrow, which are widely used in distributed data processing frameworks including Spark and DuckDB.

Modern CPUs operate on cache lines typically 64 bytes in size. In an AoS layout, accessing a single attribute often forces the processor to load an entire structure into cache, wasting memory bandwidth. In contrast, the SoA layout allows the processor to load only the required column data, maximizing useful data per cache line.

8. Related Work

Columnar storage formats are widely used in modern analytical systems because they enable efficient processing of large datasets.

Stonebraker et al. demonstrated that column-oriented database systems can significantly outperform row-based systems for analytical queries due to improved cache utilization and reduced I/O requirements (Stonebraker et al., 2005).

Similarly, vectorized query processing engines exploit contiguous column storage to improve CPU efficiency and enable SIMD optimizations.

These principles are widely used in systems such as:

- Apache Parquet
 - Apache Arrow
 - MonetDB
 - ClickHouse
-

9. Conclusion

This project investigated the performance impact of memory layout and parallelism on analytical query workloads.

The results show that:

1. Reorganizing data from AoS to SoA improved query performance by approximately 45×.
2. OpenMP parallelization improved performance but exhibited diminishing returns due to memory bandwidth limitations.
3. The SoA layout enabled significantly better parallel scaling, achieving over 5× speedup with 12 threads.

These findings highlight that memory layout optimization is often more impactful than parallelization alone for data-intensive workloads.

The experiment demonstrates that optimizing memory layout can produce significantly larger performance gains than parallelization alone. This highlights the importance of cache-efficient data structures in modern data processing systems.

References

Stonebraker, M., Abadi, D., Batkin, A., Chen, X., Cherniack, M., Ferreira, M., Lau, E., Lin, A., Madden, S., O'Neil, E., O'Neil, P., Rasin, A., Tran, N., & Zdonik, S. (2005).

C-Store: A Column-oriented DBMS.

Proceedings of the 31st International Conference on Very Large Data Bases.

New York City Taxi and Limousine Commission. (2023).

TLC Trip Record Data.

<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>